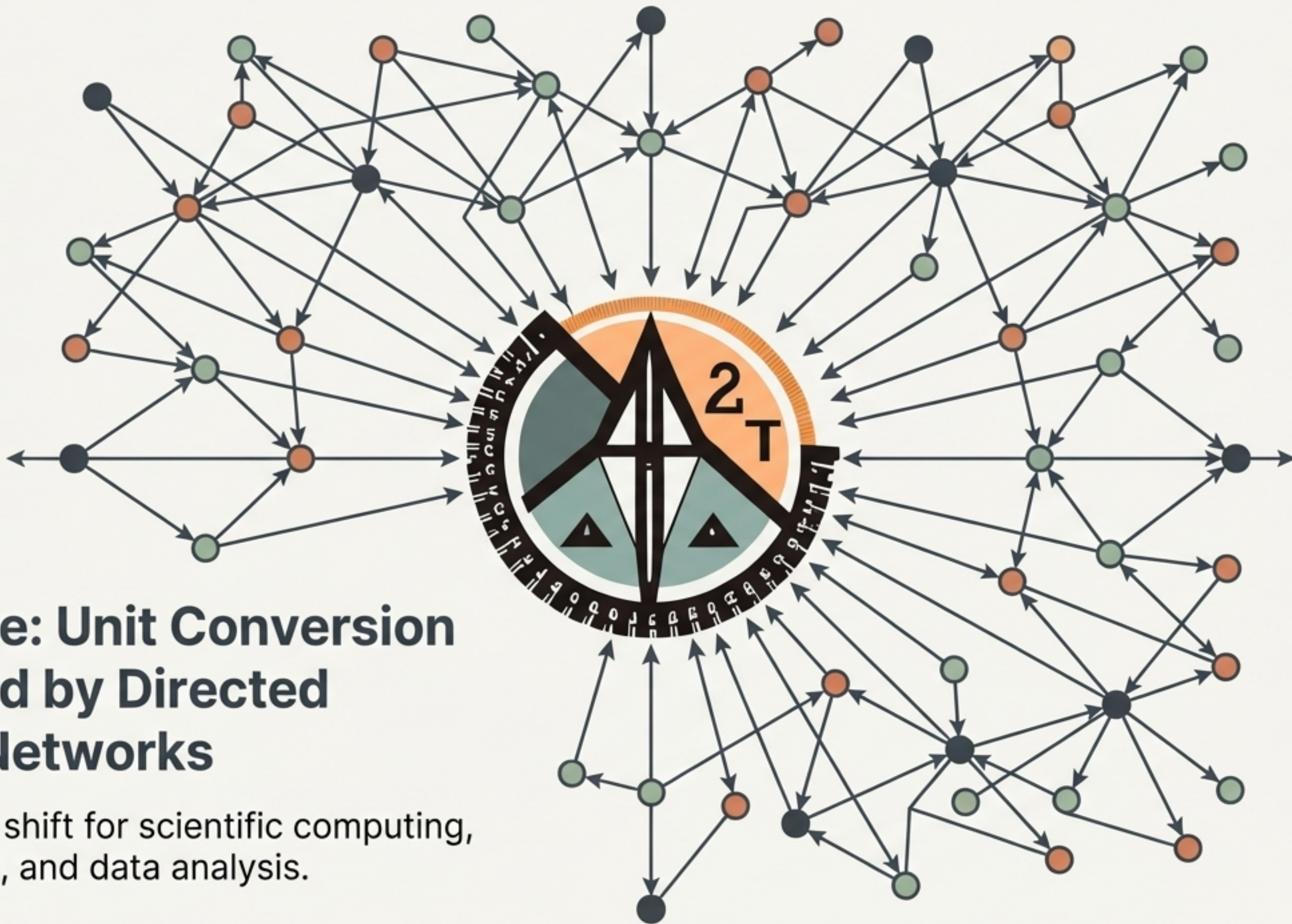




Headline: Unit Conversion Powered by Directed Graph Networks

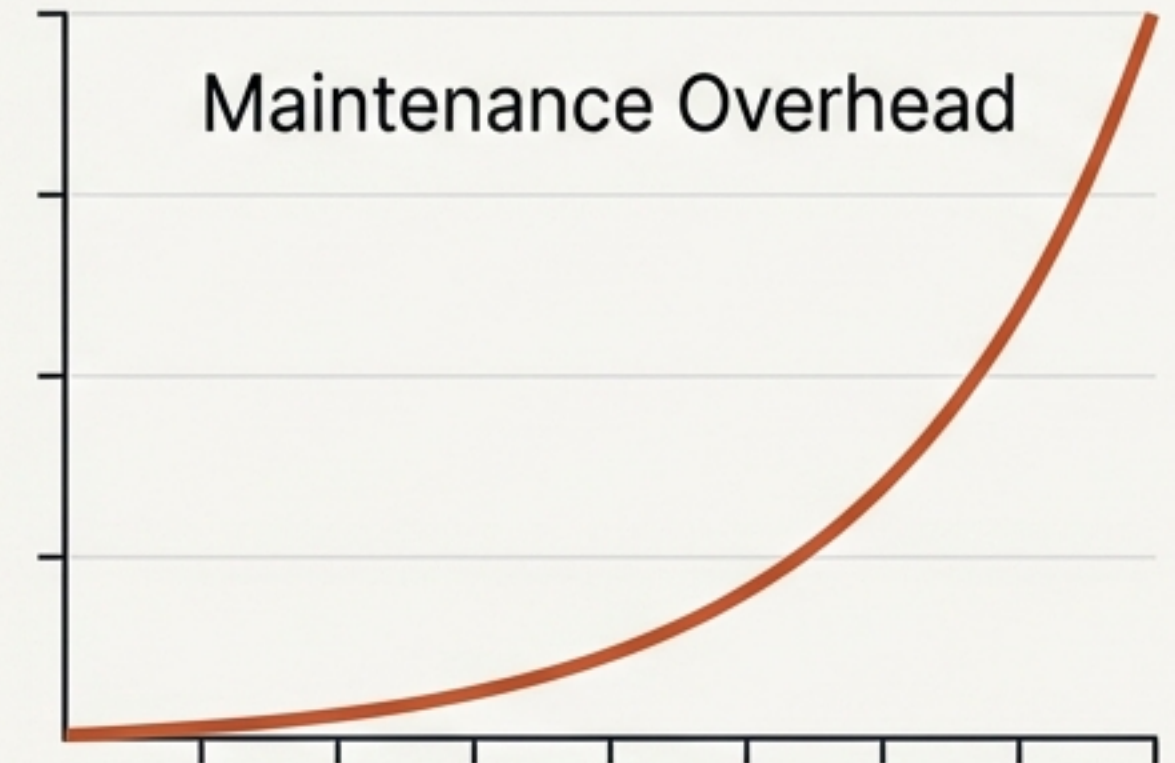
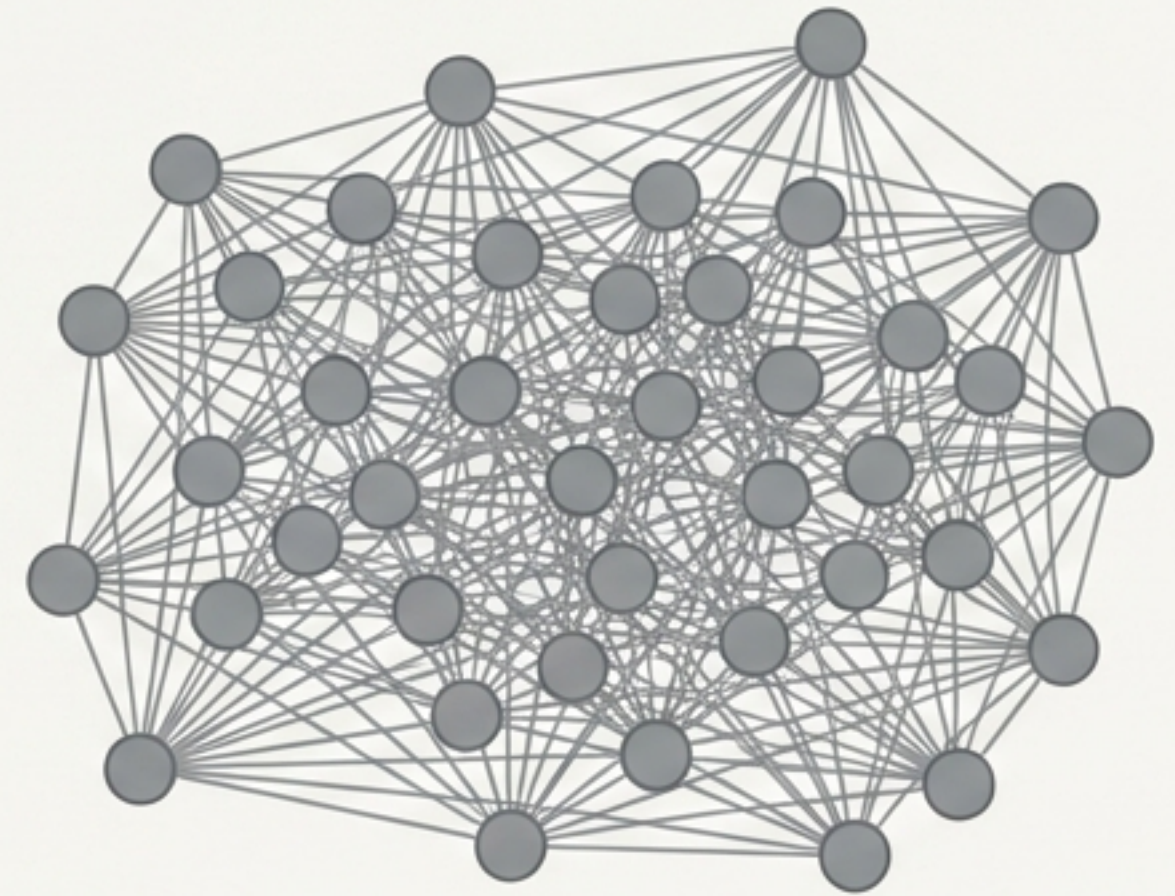
A paradigm shift for scientific computing,
engineering, and data analysis.

The $O(n^2)$ Scaling Bottleneck

Traditional converters rely on exhaustive lookup tables.

Adding a single new unit requires defining conversions for every existing unit in the system.

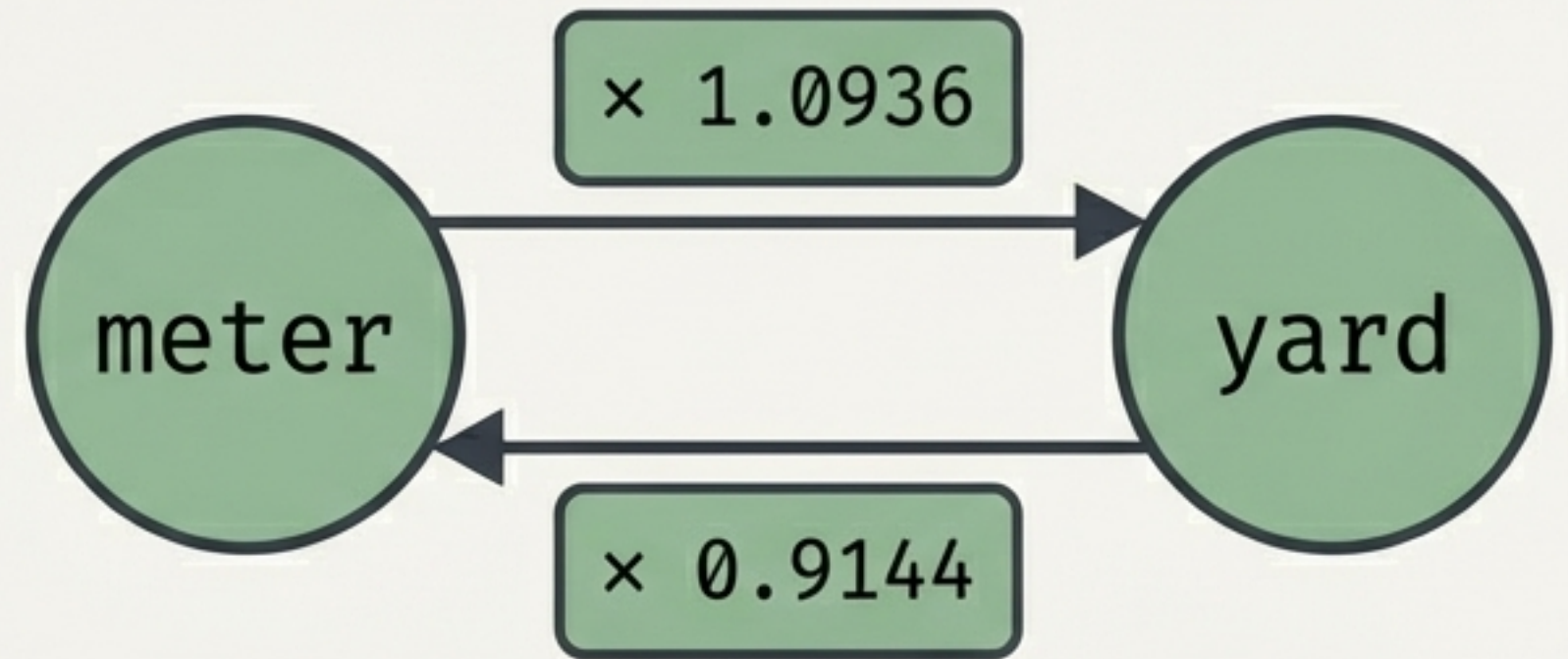
Ratios like kg/m^3 or $\text{stb}/\text{day}/\text{psi}$ cause these rigid tables to explode exponentially in memory and maintenance overhead.



Rethinking Conversions as a Graph Network

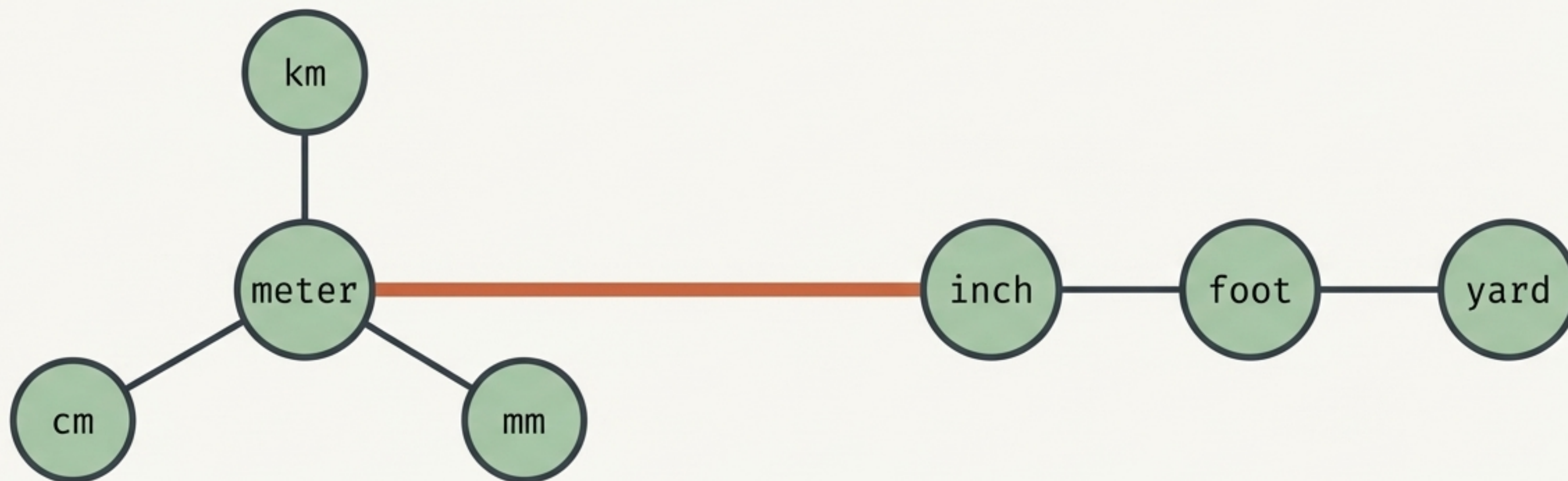
unyts models units as vertices (UNode) and conversion rules as directed edges.

Adding a new unit now requires only 1 to 2 edges to connect it to the network, scaling elegantly at $O(n)$.



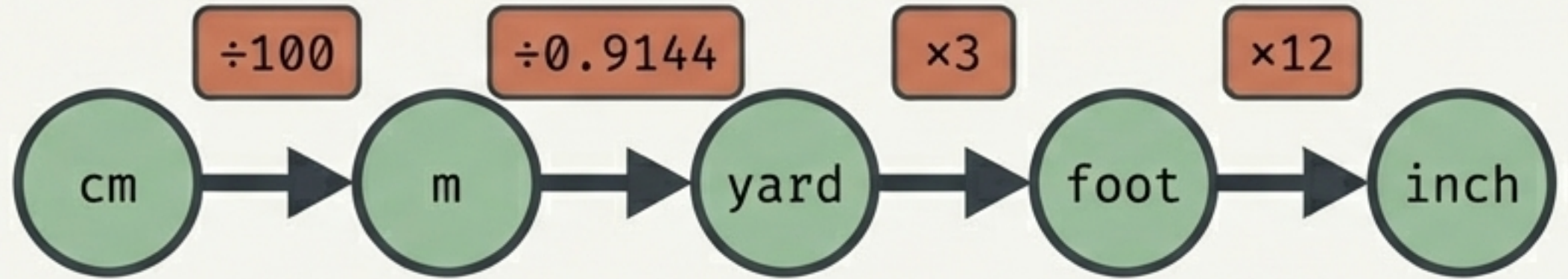
Network Topology: Canonical Connections

There are no duplicate paths. A single canonical bridge links the SI and Imperial systems, preventing conflicting conversion factors and the accumulation of floating-point rounding errors.



Anatomy of a Conversion

1. Discover: The algorithm finds the path.
2. Transform: Mathematical rules are applied at each edge.
3. Cache: The final multiplier (0.3937) is cached in memory, ensuring subsequent lookups are instantaneous $O(1)$ operations.



The Routing Engine

BFS



Layer-by-layer exploration.
Guarantees the absolute shortest path.

lean_BFS



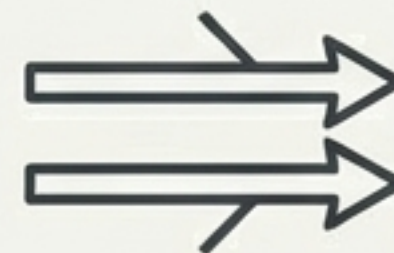
The default.
Prunes the network for speed.

DFS

Plunges deep into branches.
Ideal for complex compound ratios.



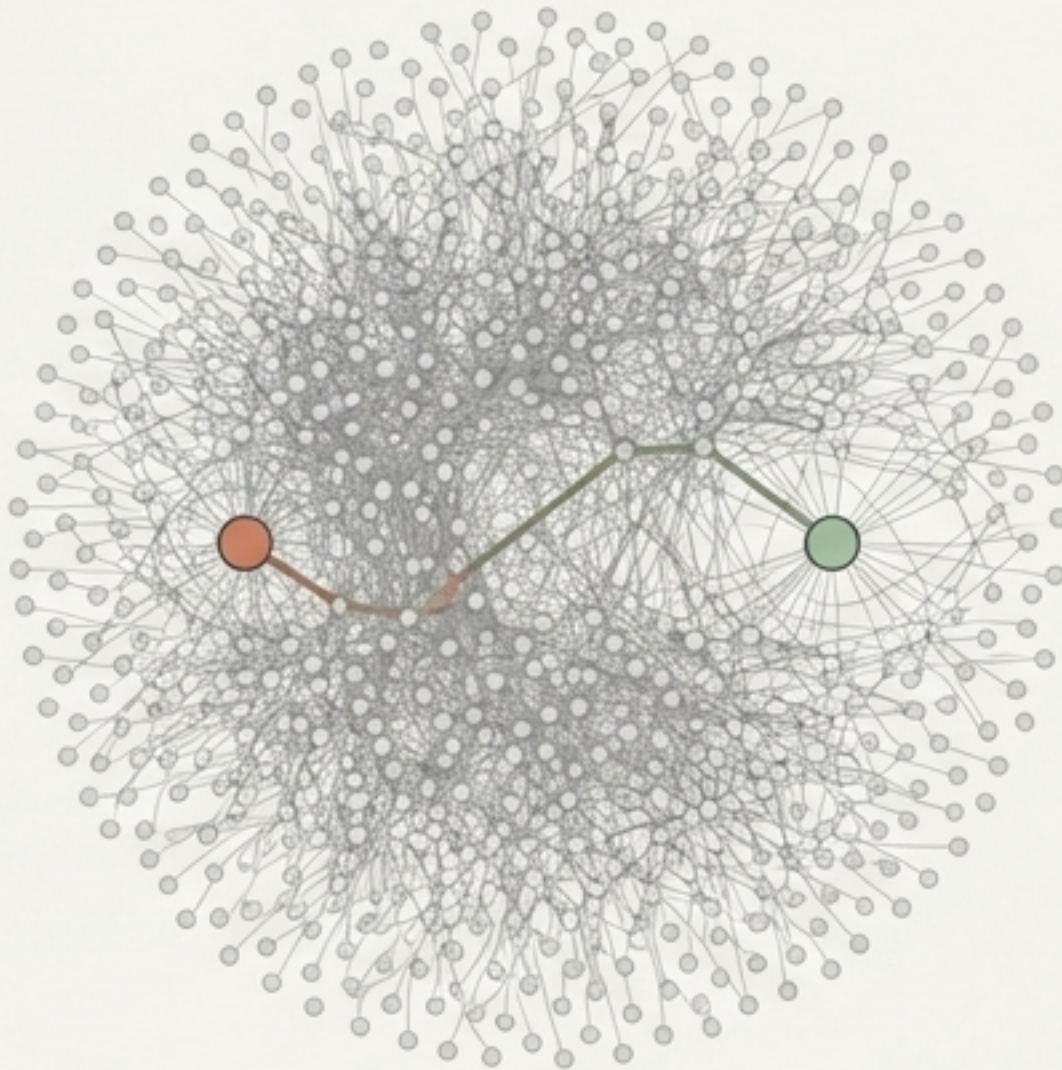
hybrid_BFS



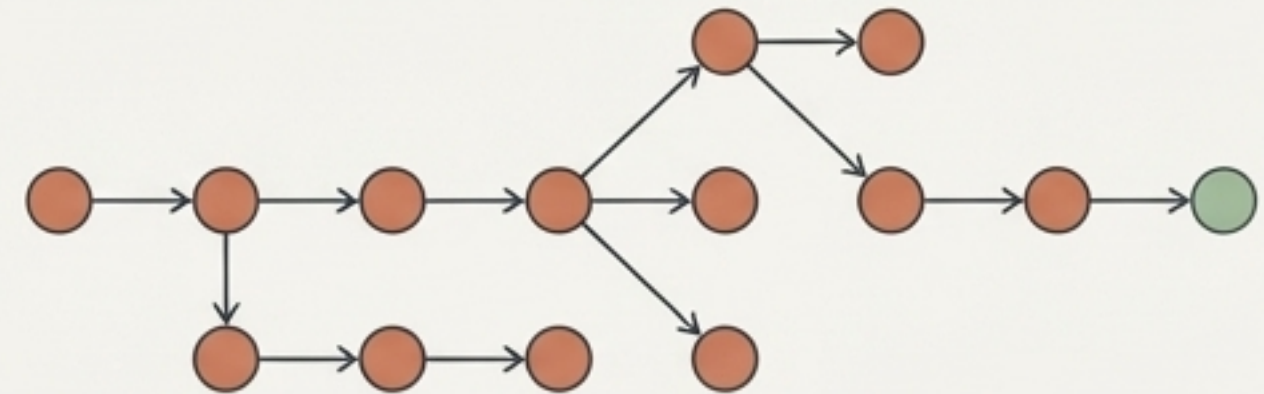
Executes BFS and lean_BFS
in parallel across multiple
cores for raw speed.

Optimised Pathfinding with Lean_BFS

Standard searches waste time on dead ends. Lean_BFS pre-computes descendants for both the start and end nodes, finds the intersection, and builds a slimmed-down network. The result is 5–20x faster performance while maintaining shortest-path guarantees.

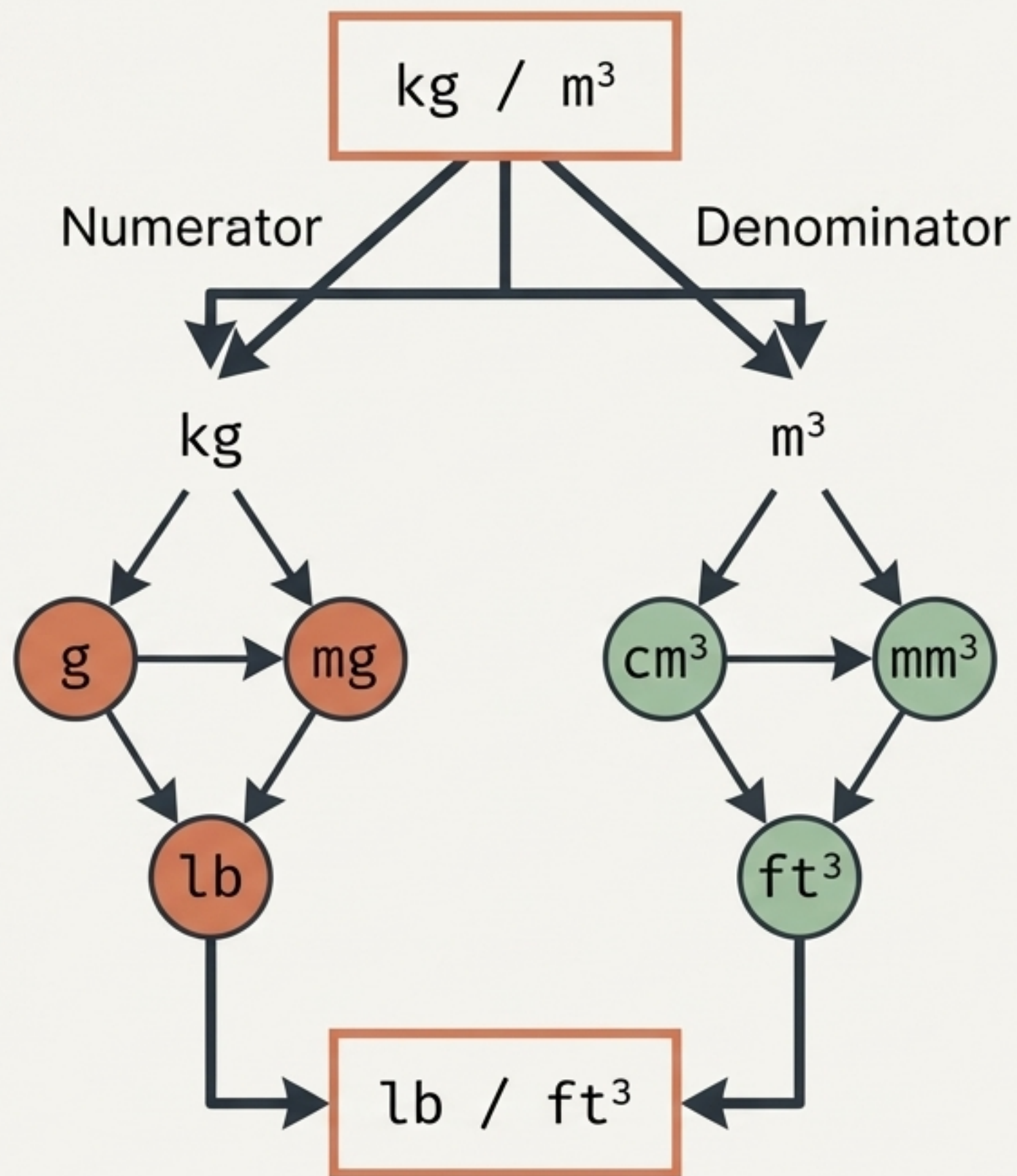


Before & After



Natural Composability for Complex Ratios

There is no need to manually define compound units. unyts intrinsically understands ratios, decomposing elements to traverse the graph independently and recombining them perfectly at the destination.



Intuitive Unit-Aware Calculations

The Unit class handles implicit conversions mid-calculation. It natively supports addition, multiplication, exponentiation, and logical comparisons without forcing the developer to manually align base metrics first.

```
from unyts import units

length = units(6, 'in') + units(1, 'ft')
print(length) # Outputs: 18_in
```


Seamless Data Science Integration

Built for data engineering at scale. unyts natively parses and wraps numpy arrays and pandas DataFrames, enabling bulk, unit-aware conversions across massive datasets.

NUMPY INTEGRATION

```
# NumPy array integration
distances = units(np.array([1, 2, 3]),
                    'km')
distances.to('mi')
```

PANDAS INTEGRATION

```
# Pandas Series integration
series = pd.Series([10, 20, 30])
wrapped = units(series, 'kg')
wrapped.to('lb')
```


Advanced Engineering Workflows

Move across incompatible dimensions. Setting a density parameter automatically bridges mass and volume graphs. Setting an FVF enables complex reservoir-to-surface volume conversions for the petroleum industry.

Constant Density Bridge

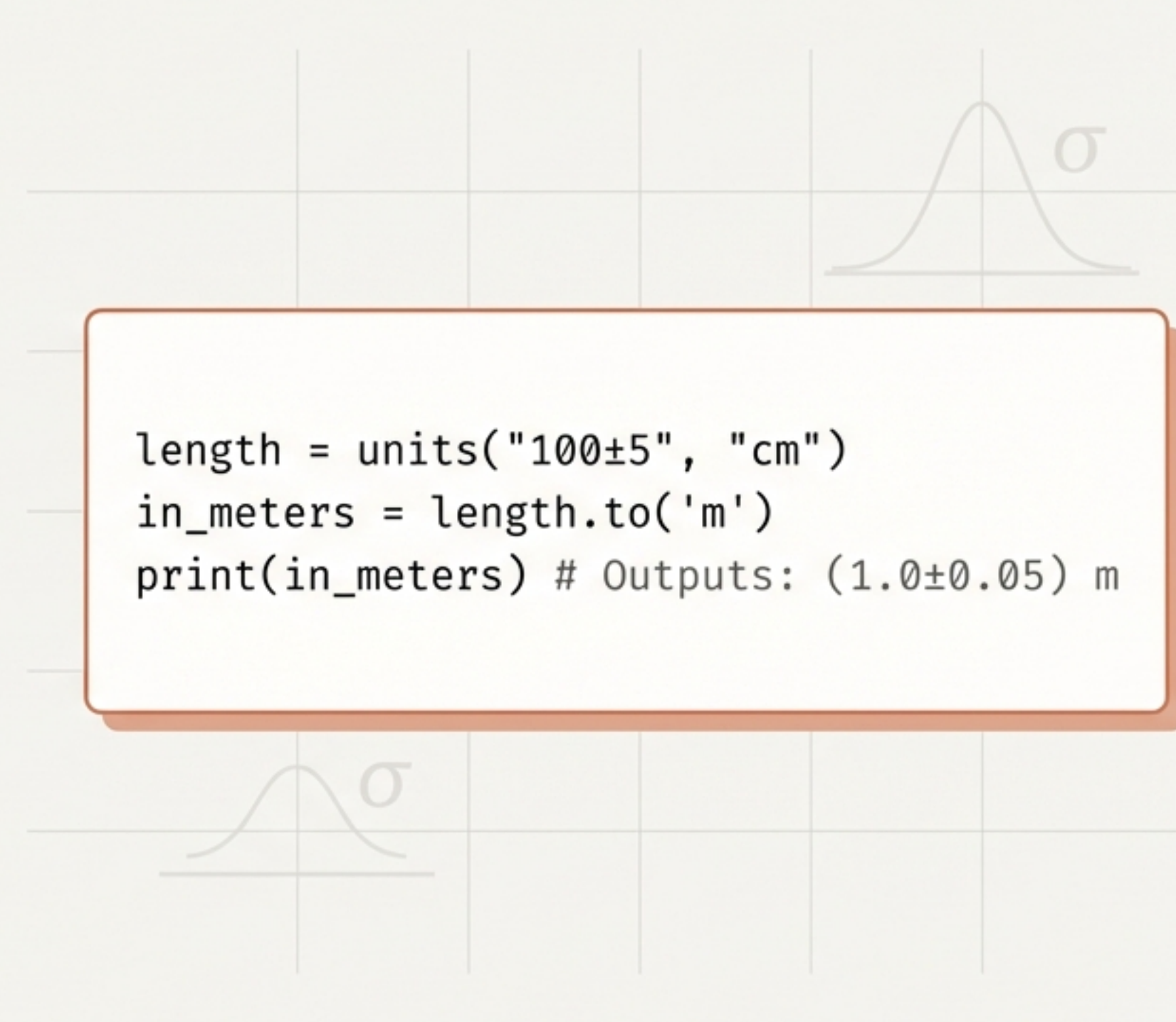
```
unyts.set_density(997)  
# Automatically bridges volume to mass
```

Formation Volume Factor

```
units_network.set_fvf(1.25)  
# Converts rb directly to sm3
```


Preserving Measurement Uncertainty

Real-world data carries error margins. The framework natively parses uncertain units, preserving standard deviations accurately through both complex arithmetic operations and graph traversals.

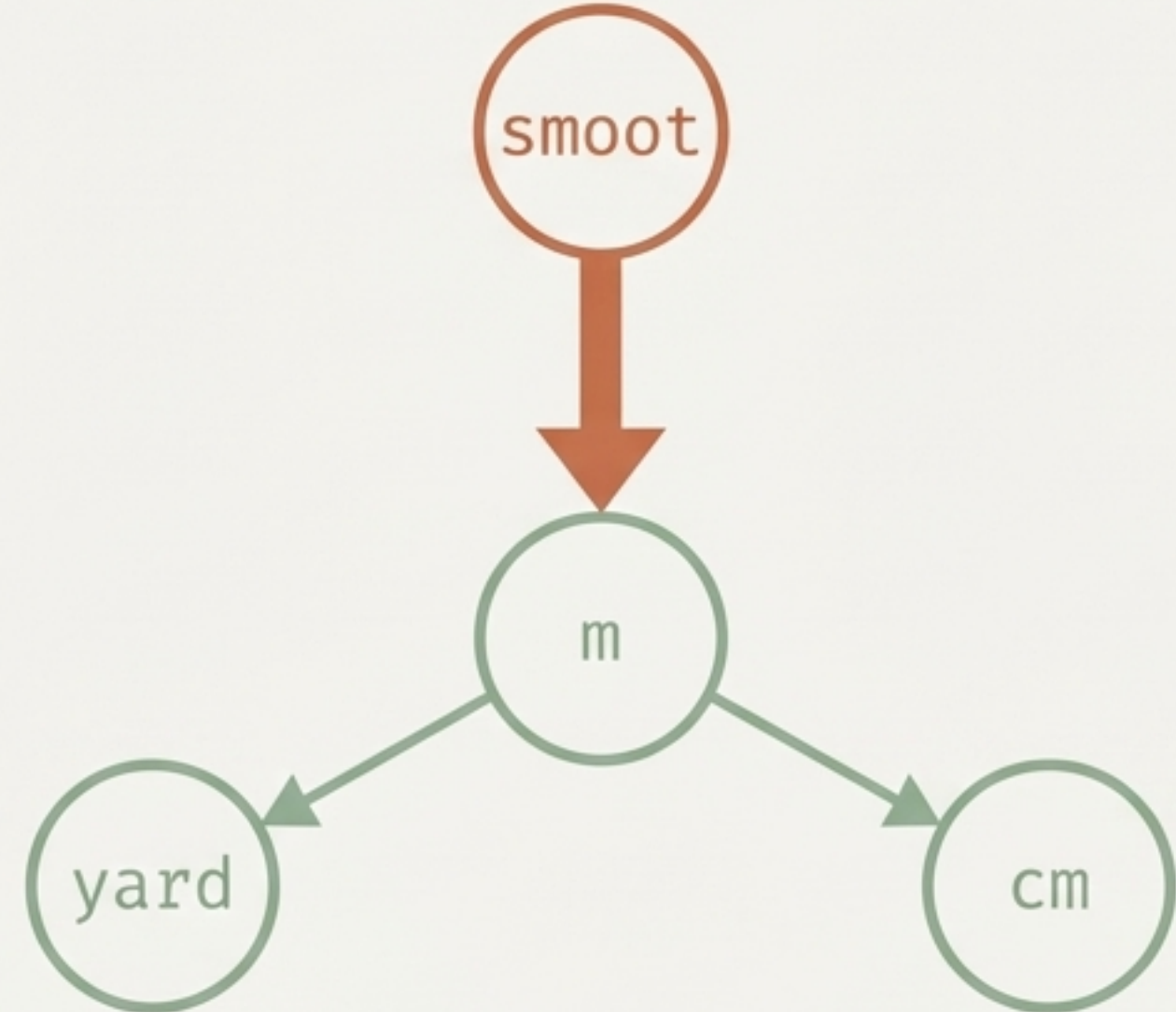


```
length = units("100±5", "cm")  
in_meters = length.to('m')  
print(in_meters) # Outputs: (1.0±0.05) m
```


Instant Framework Extensibility

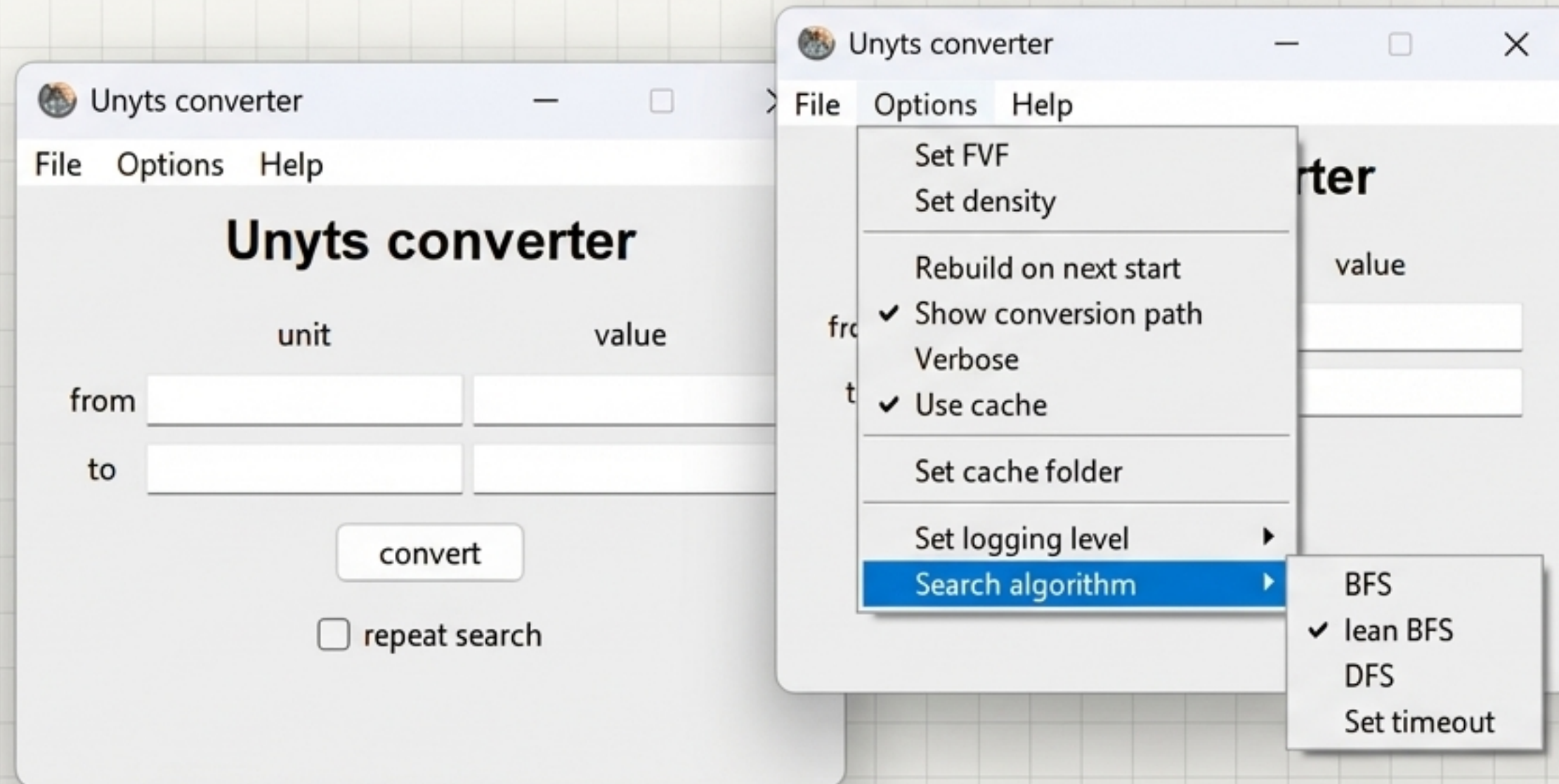
Add custom units to the database in seconds. Once defined relative to a single existing node, the custom unit instantly participates in all graph search operations across the entire network.

```
from unyts.database import units_network  
  
units_network.set_unit('smoot', 1.7018, 'm')
```



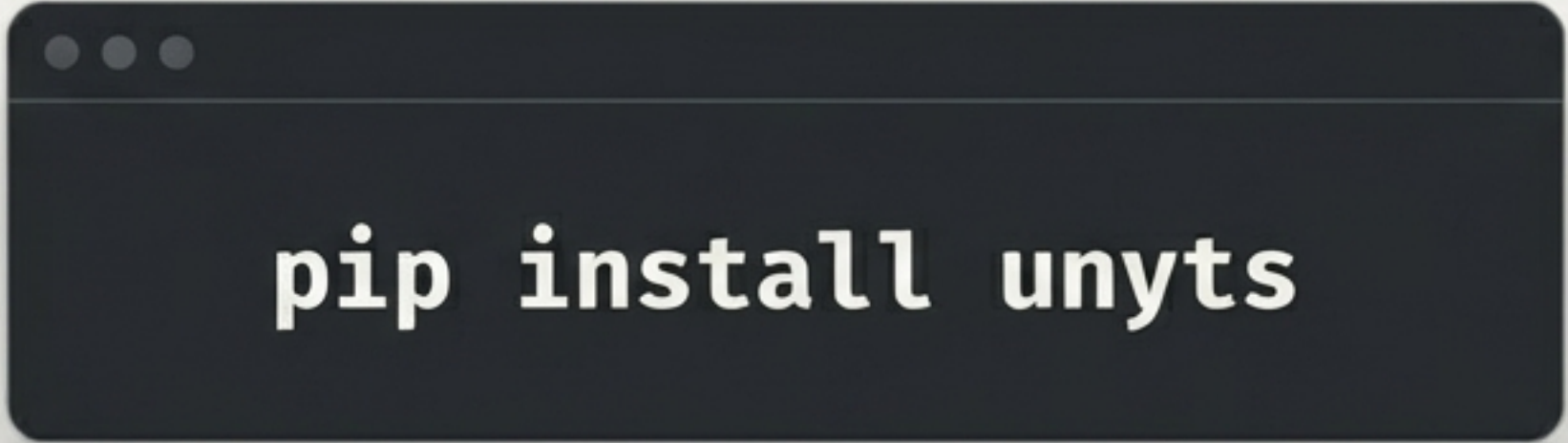
The Standalone Graphical Interface

Launch a bidirectional conversion tool directly from the terminal with 'python -m unyts'. It features persistent memory caching, algorithm switching, and full access to density and FVF bridges without writing a line of code.

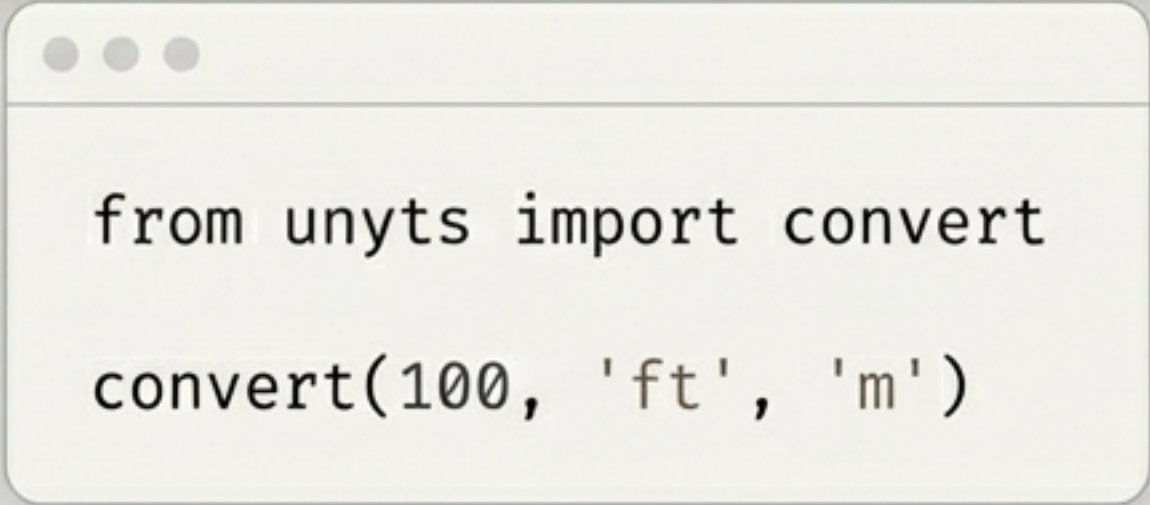


Get Started

Requires Python 3.7+. Zero mandatory external dependencies. Install from PyPI today and explore the Jupyter demo repository to master unit-aware calculations.



```
pip install unyts
```

A dark-themed terminal window with three window control buttons in the top-left corner. The text `pip install unyts` is displayed in a bold, white, monospaced font.

```
from unyts import convert  
convert(100, 'ft', 'm')
```

A light-themed terminal window with three window control buttons in the top-left corner. The text `from unyts import convert` and `convert(100, 'ft', 'm')` is displayed in a monospaced font.